

Phase 2 now begins on the roadmap. Deadline was September 1st for the AD project, so I'm quite far ahead now.

Phase 2 = Security+, Linux, Offensive Security Foundations

Today: Learn Security Theory -> Learn the operating environment -> learn how professionals assess systems -> apply it to my lab

Security+ Lesson 1

A security control is a safeguard used to help manage security risk. I've already implemented many controls without necessarily calling them that.

Examples: Windows Firewall, SMB Signing, NTFS permissions, Password policy, Windows Security Auditing

However, they don't all do the same thing. Security+ therefore gives us different ways to classify controls. The first distinction is known as: Control category.

This answers: What kind of control is this? There are four categories:

1. Technical: Technology performs or enforces the control
 - a. Examples: firewall, authentication system, encryption, NTFS permissions, antivirus, SMB signing
 - b. Alice HR test is a perfect example. Alice accessed \\SRV-DC01\Finance. Windows evaluated her permissions and returned: Access Denied. A human didn't manually make that decision. The technology enforced it. That's a technical control.
2. Managerial: Managerial controls deal primarily with: governance, policies, risk management, standards, and planning
 - a. Imagine: Bear Corp decides: Administrative accounts must not be used for ordinary daily work. That's an organizational security requirement. The actual technology enforcing parts of that requirement may be technical, but the policy itself is managerial.
3. Operational: Operational controls involve the procedures and actions people perform as part of security operations.
 - a. Examples: security-awareness training, incident-response procedures, manual access reviews, analysts reviewing alerts, personnel performing backups
 - b. Think: Humans carrying out security processes
4. Physical: Controls protecting physical access or environments
 - a. Examples: locked server rooms, fences, badge readers, guards, bollards

Good mental model:

Technical: "What TECHNOLOGY enforces security?"

Managerial: "What does the ORGANIZATION require/manage?"

Operational: "What do PEOPLE/PROCESSES do?"

Physical: "What PHYSICALLY restricts/protects something?"

Control Functions, different question: "What is the control trying to accomplish?"

1. Preventive: attempts to prevent an unwanted event
 - a. Authorization Structure: Alice HR -> Finance Share -> Access DENIED
 - i. Preventive! It stopped unauthorized access.
 - b. Firewall can also be preventive when it blocks traffic.
2. Detective: helps identify that activity happened
 - a. Failed login -> 4625 created -> Analyst sees evidence...Event log didn't stop the failed authentication attempt. It gave visibility.
3. Corrective: helps correct a situation after something has gone wrong
 - a. Example: Account compromised -> Disable account -> Reset credentials
 - i. Or restoring damaged data from backups
4. Deterrent: discourages someone from attempting undesirable behavior
 - a. Example: AUTHORIZED USERS ONLY, ACTIVITY IS MONITORED
 - b. It isn't physically preventing someone from trying; it's intended to discourage the attempt
5. Directive: tells people what they are expected or required to do
 - a. Example: Report suspected phishing messages to the security team.
 - b. You are directing behavior.
6. Compensating: used when the preferred security control isn't possible, so another control helps compensate for the missing protection.
 - a. Suppose some ancient application cannot support MFA. You might compensate using: stricter network location, extremely limited access, additional monitoring
 - i. You haven't magically made MFA unnecessary; instead, you've reduced risk another way

A control can have: CATEGORY + FUNCTION. Windows Firewall is Technical + Preventive. Windows Event Logging is Technical + Detective.

Asset: something valuable that needs protection. Examples from lab: SRV-DC01, Active Directory credentials, HR files, Finance files, User accounts

Vulnerability: A weakness that could potentially be abused. Example: Minimum password length = 5. That's a weakness in authentication posture. That's the vulnerability.

Threat: Something capable of causing harm. Example: attacker attempting credential guessing. The attacker/activity is the threat. The weak password configuration is the vulnerability.

Risk: concerns the possibility of a threat exploiting a vulnerability and the consequences if that happens. Asset + Vulnerability + Threat -> Potential Negative Outcome -> Risk

Two basic ideas with risk: Likelihood and Impact

Likelihood is how likely is the event, Impact is how damaging would the outcome be?

Control: so, a safeguard is introduced which is intended to reduce that risk

Example: Asset = Domain credentials, Vulnerability = weak authentication configuration, Threat = password guessing, Risk = unauthorized domain access, Controls = Stronger authentication requirements, monitoring, lockout/rate-limiting strategy, MFA

Security controls generally don't eliminate all risk. Imagine you implement: strong passwords + MFA + authentication monitoring. Can an account never be compromised? No. There's still some remaining risk called residual risk. Initial Risk -> Security Controls -> Reduced Risk -> Residual Risk

Linux Foundations For Cybersecurity

Linux Filesystem

On Windows we have things such as C:\ C:\Users\Kevin C:\Windows

But on Linux, everything begins from /. This is called the root directory; not the root user.

Today we are going to focus primarily on /home. This is user home directories.

My Kali account is /home/nigist which is comparable to C:\Users\Kevin on Windows.

/etc: configuration files for the operating system and many applications. Security relevance: if you're investigating or administering Linux, configuration often matters enormously

/var: data that changes while the system operates (logs, application data, caches)

/tmp: temporary storage

Absolute vs Relative Paths:

Suppose you're here: /home/kali....An absolute path begins from /.

Ex: /home/kali/Documents/report.txt....This identifies the full location. A relative path; however, will start from your current location. If already in: /home/kali, you might refer Documents/report.txt. This is the same file as in the absolute path. Just a different way of specifying it.

Users and Identity

Run: whoami

Next, try the command: id. (Gives info such as user ID, group ID, and groups)

Linux Permissions

Use the command: ls -l

See output such as: -rwxr-xr-

Temporarily ignore the first character: rwx | r-x | r - -

These correspond to: Owner | Group | Others

R = read, W = write, X = execute

Rwx = read, write, execute

R-x = read and execute but not write

R - - = Read only

We can also introduce the number system for this: r = 4, w = 2, x = 1

So: rwx = 7, r-x = 5, r-- = 4...meaning 754 can describe Owner: rwx, Group: r-x, Others: r--

Processes

Process: running instance of a program, Linux assigns process a: PID (Process ID)
Inspect processes using the command: ps and ps aux.

Linux Networking:

Command: ip addr, ip route (routing information), ss (socket/network connection information)

Offensive Security Foundations

So with Phase 2, I want to understand how a real assessment begins. One of the biggest mistakes beginners make is thinking pentesting means: Run nmap -> google exploit -> run exploit -> "pentest complete"

Stage 0: Authorization and Scope

Before touching anything: What am I authorized to assess? As an example, in my lab:
Authorized: SRV-DC01, CLIENT01
Not automatically authorized: Every device on my home network

Stage 1: Reconnaissance

Reconnaissance means gathering information about the target. The question is: What can I learn about this environment?

Stage 2: Host Discovery

Now we might ask: Which systems are actually reachable? Imagine I am given: 192.168.1.0/24, there may be many possible addresses. Which addresses actually correspond to live hosts?

Stage 3: Port Enumeration

Once we know a host exists: What network ports appear accessible? Ex: 22, 80, 443, 445

Stage 4: Service Enumeration

Then: what is actually behind that port? For example we know: TCP/445 open. But knowing we're dealing with SMB tells us much more. Even better: What implementation/version/configuration can we determine?

Stage 5: Vulnerability Analysis

Now: Does what we've discovered contain an actual weakness? (OPEN SERVICE DOES NOT EQUAL VULNERABILITY)

Stage 6: Exploitation

After identifying a vulnerability, attempt exploitation. This means taking advantage of the weakness.

Stage 7: Post-Exploitation

Now we need to determine: what access was obtained, which identity we're running as, available privileges, reachable systems, sensitive information, whether privilege escalation is possible.

Stage 8: Reporting and Remediation

Assessment needs to answer: What did you find? Why does it matter? How did you validate it? What is the impact? How should it be fixed? Did the fix work?

Guided Practical: Viewing AD Lab From Kali

We're going to:

1. Identify Kali's current network information
2. Identify the current addresses of your two Windows systems
3. Verify that Kali can reach them
4. Perform basic Nmap service enumeration
5. Look at what appears on SRV-DC01
6. Connect those exposed services back to what I built in project 1

Step 1: Understand Kali's Network Position

After running `ip addr: inet 192.168.1.238/24`

Also run: `ip route` (this shows where network traffic is routed)

Ip SRV-DC01: 192.168.1.236

IP CLIENT01: 192.168.1.196

Now to verify reachability, we ping these machines from Kali.

Ping `-c 4 <IP_address>`

SRV-DC01 is reachable, but CLIENT01 is not...due to firewall blocking ICMP echo requests.

Nmap scan: What TCP ports appear open on this target? Nmap `<IP_address>`

```
Host is up (0.013s latency).
Not shown: 989 filtered tcp ports (no-response)
PORT      STATE SERVICE
53/tcp    open  domain
88/tcp    open  kerberos-sec
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
389/tcp   open  ldap
445/tcp   open  microsoft-ds
464/tcp   open  kpasswd5
593/tcp   open  http-rpc-epmap
636/tcp   open  ldapssl
3268/tcp  open  globalcatLDAP
3269/tcp  open  globalcatLDAPssl
```

PORT | STATE | SERVICE

Open = accepting connections there

Nmap -sV does service/version detection